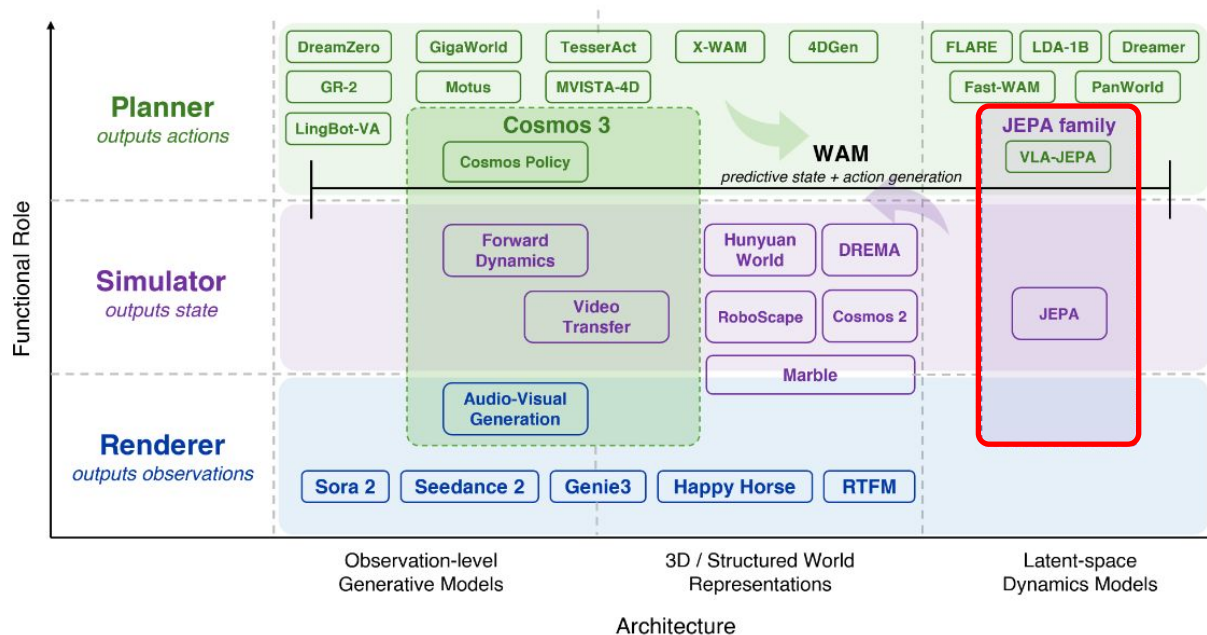


LeWorldModel: **Stable End-to-End** Joint-Embedding Predictive Architecture from Pixels

New York University, 2026.03



Two-dimensional Taxonomy of Representative World Models



JEPA식 World Model: 현재 상태와 행동을 바탕으로 미래 상태가 어떻게 변할지 예측하는 내부 시뮬레이터



What are the **limitations** of LeWM?

Can we **contribute** to this subject?



O_t



O_{t+1}

Contents

1. Introduction
2. Motivation
3. Methods
4. Results
5. Limitations
6. Ideation

Contents

1. Introduction
2. Motivation
3. Methods
4. Results
5. Limitations
6. Ideation

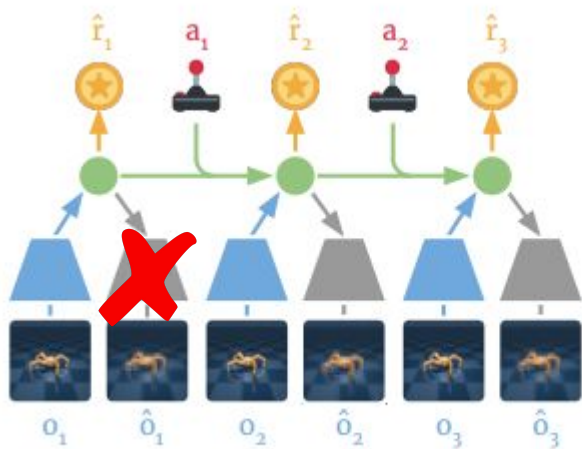
Background

기계는 어떻게 인간과 동물처럼 **효율적**으로 학습할 수 있을까?

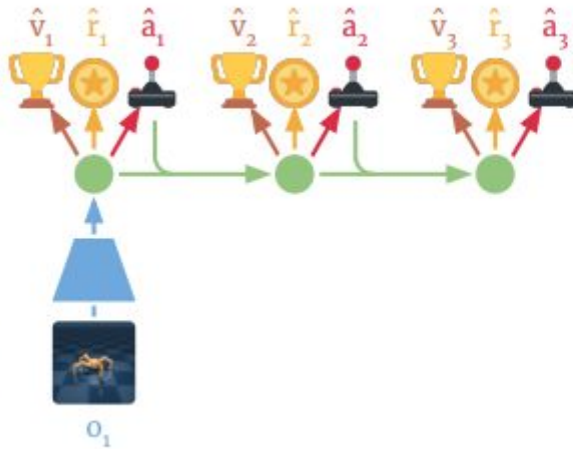
기계는 어떻게 **추론하고 계획하는 법**을 배울 수 있을까?

어떻게 해야 기계가 다양한 추상화 수준에서 지각과 행동 계획의 표현을 학습하여,
여러 시간 지평에 걸쳐 추론, 예측, 계획을 할 수 있게 될까?

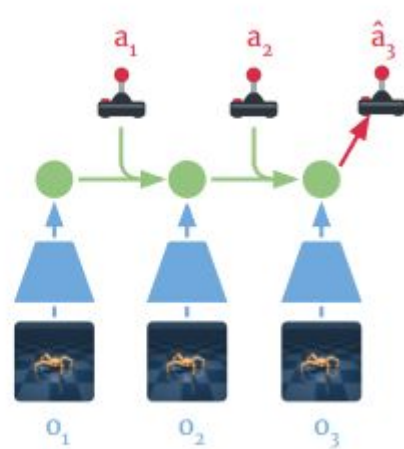
Dreamer (WM, RL)



(a) Learn dynamics from experience



(b) Learn behavior in imagination

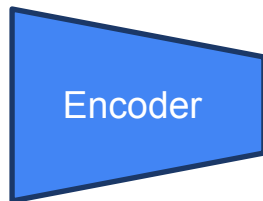


(c) Act in the environment

“task와 무관한 details를 학습하게 만드는
pixel reconstruction loss는 불필요하다.”

Latent space loss만 사용

[1,0.3,0.4,-0.2]
[2.3,0.5,-0.2,3]
[1.2,3.4,-1.5,5]
[1,0.5,0.2,-1.5]

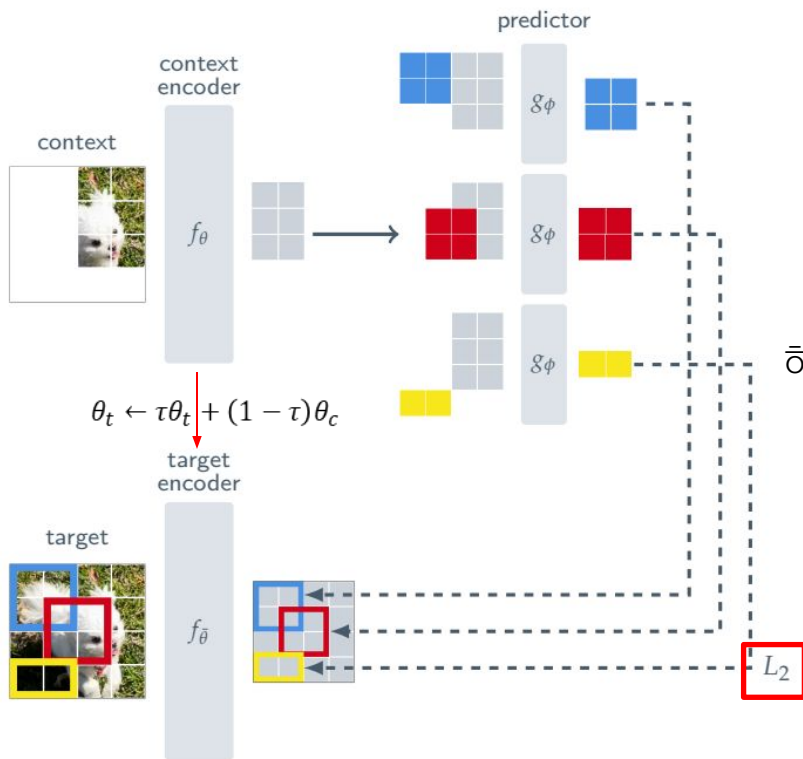


[0.10,
0.10]
[0.11, 0.09]
[0.10,
0.11]
[0.11, 0.12]

Representation collapse 시작.

Exponential Moving Average: 현재 값을 조금씩만 반영하면서
과거 값의 흐름을 부드럽게 따라가는 지수 가중 평균
(타겟 파라미터를 보수적으로 업데이트)

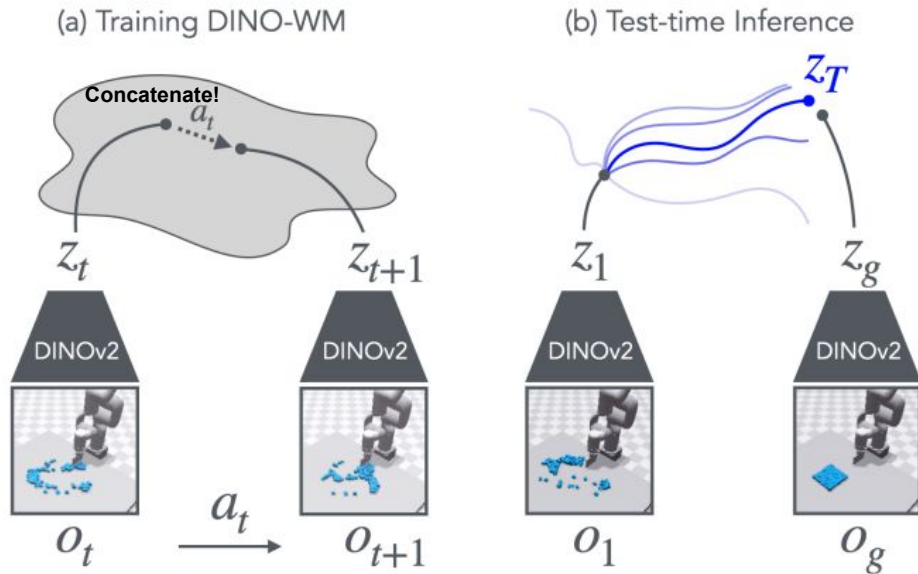
I-JEPA



“**latent space loss 사용** 및
EMA로 **representation collapse 완화**”

하지만 EMA는 representation collapse를 완화하는 이유에 대한
수학적 근거가 없다.

DINO-WM



Frozen pretrained vision encoder로 representation collapse 완화
하지만 수학적 근거가 없다.

PLDM (WM)

$$\mathcal{L}_{\text{JEPA}} = \mathcal{L}_{\text{sim}} + \alpha \mathcal{L}_{\text{var}} + \beta \mathcal{L}_{\text{cov}} + \delta \mathcal{L}_{\text{time-sim}} + \omega \mathcal{L}_{\text{IDM}}$$

Regularization terms(VICReg)로 representation collapse 완화
수학적 근거는 있다.

다만 Hyperparameter가 많아
최적의 Hyperparameter를 구하기 어렵다. ($O(n^6)$)

LeJEPA

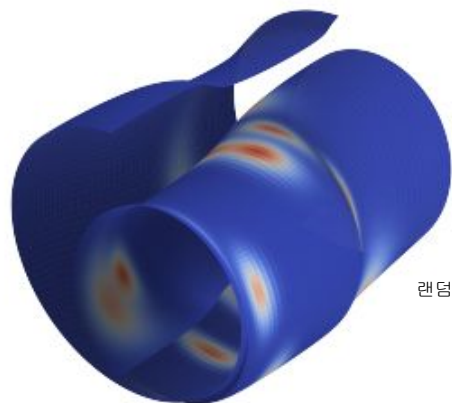
Regularization term(**SIGReg**)로 **representation collapse(RC)** 완화
수학적 근거가 있다.

SIGReg (Regularization Term)

- Latent embedding이 collapse되지 않도록, 표현 분포를 isotropic Gaussian에 가깝게 강제하는 anti-collapse regularizer
- Isotropic Gaussian(등방성 가우시안):
모든 방향으로 똑같이 퍼진 가우시안 분포
 - 각 차원의 분산이 모두 같음.
 - 차원간의 공분산이 0임.
 - $z \sim N(0, I)$ (variance = 1)

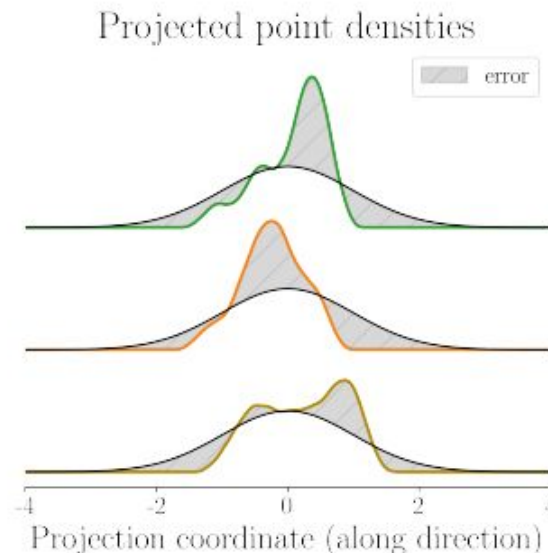
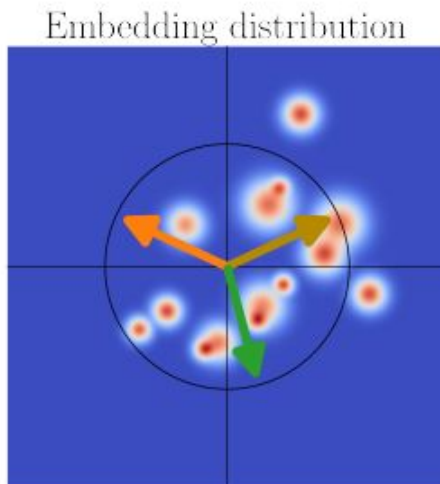
SIGReg (Regularization Term)

Cramer-Wold 정리: 모든 방향에 대한 1D projection이 Gaussian이면, 전체 벡터 X 도 가우시안이다.



분포가 p_x 인 input data

f_θ
→
랜덤 1D 방향으로 projection



$$\text{Loss} = (\cos_mean - \mathbf{\phi})^2 + \sin_mean^2$$

등방성 가우시안 분포에
맞춘다.

SIGReg (Regularization Term)

$$\text{SIGReg}(\mathbf{Z}) \rightarrow 0 \iff \mathbb{P}_{\mathbf{Z}} \rightarrow N(0, \mathbf{I}).$$

$$\text{SIGReg}(\mathbf{Z}) \triangleq \frac{1}{M} \sum_{m=1}^M T^{(m)},$$

$$T^{(m)} = \int_{-\infty}^{\infty} w(t) \left| \phi_N(t; \mathbf{h}^{(m)}) - \phi_0(t) \right|^2 dt,$$

$$\phi_N(t; \mathbf{h}) = \frac{1}{N} \sum_{n=1}^N e^{it\mathbf{h}_n}$$

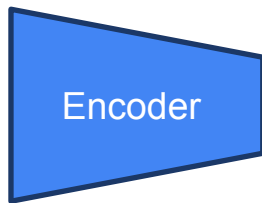
SIGReg (Regularization Term)

- 등방성 가우시안 분포에 맞춘다 = Complete collapse & Dimensional collapse X

모든 입력이 동일한 벡터로 매핑 X

임베딩이 저차원 부분공간에만 분포 X

[1,0.3,0.4,-0.2]
[2.3,0.5,-0.2,3]
[1.2,3.4,-1.5,5]
[1,0.5,0.2,-1.5]



[0.10, 0.52]
[0.43, 0.01]
[0.70, 0.15]
[0.55, 0.14]

LeWM은 LeJEPa의 SIGReg를 적용한 WM이다.
이를 통해 RC를 완화하고 prior works보다 더 효율적이다.

Contents

1. Introduction
2. Motivation
3. Methods
4. Results
5. Limitations
6. Ideation

Limitations of Prior Works

Model	Limitations
PLDM	• 6 개의 하이퍼파라미터 필요
DINO-WM	• 동결된 사전 학습 인코더 사용 • end-to-end 아님
Dreamer	• 특정 태스크에 국한됨 • 이미지 재구성 필요
TD-MPC	• 특정 태스크에 국한됨 • 보상 재구성 필요

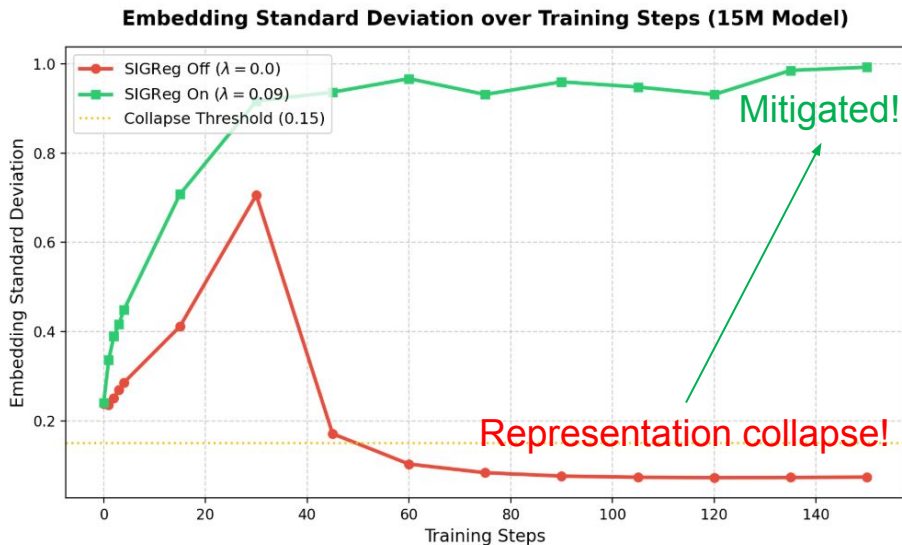
Need to make the model **fast & simple without representation collapse!**

Contents

1. Introduction
2. Motivation
3. **Methods**
4. Results
5. Limitations
6. Ideation

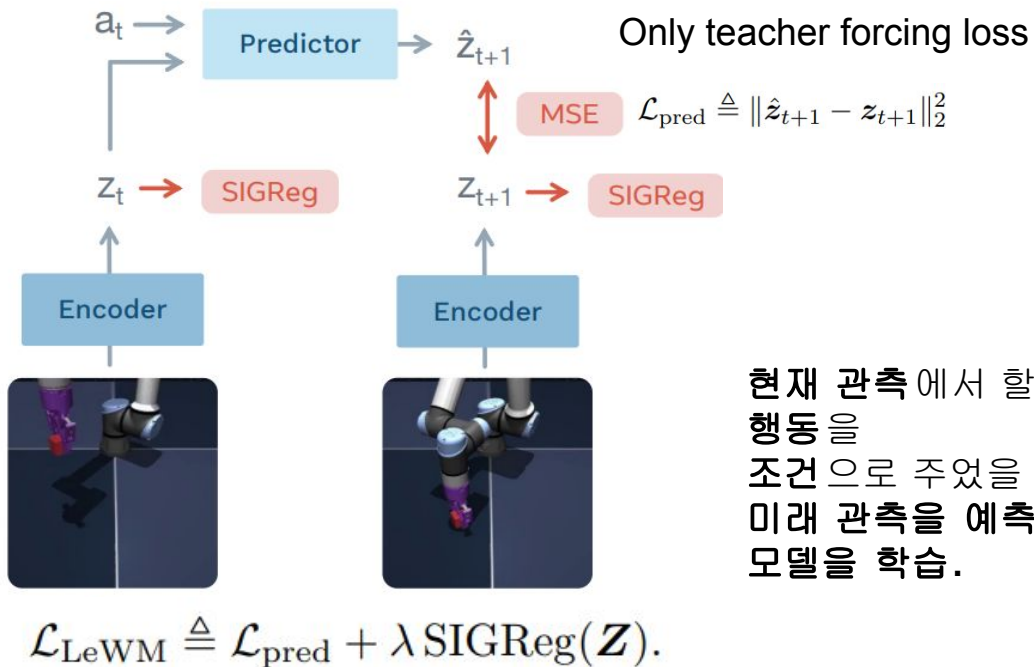
SIGReg did everything!

- VICReg(6 hyperparameters) -> SIGReg (1 hyperparameter (성능에 민감한))
- SIGReg -> Prevent Collapse -> No need Frozen Pretrained Encoder
-> end-to-end



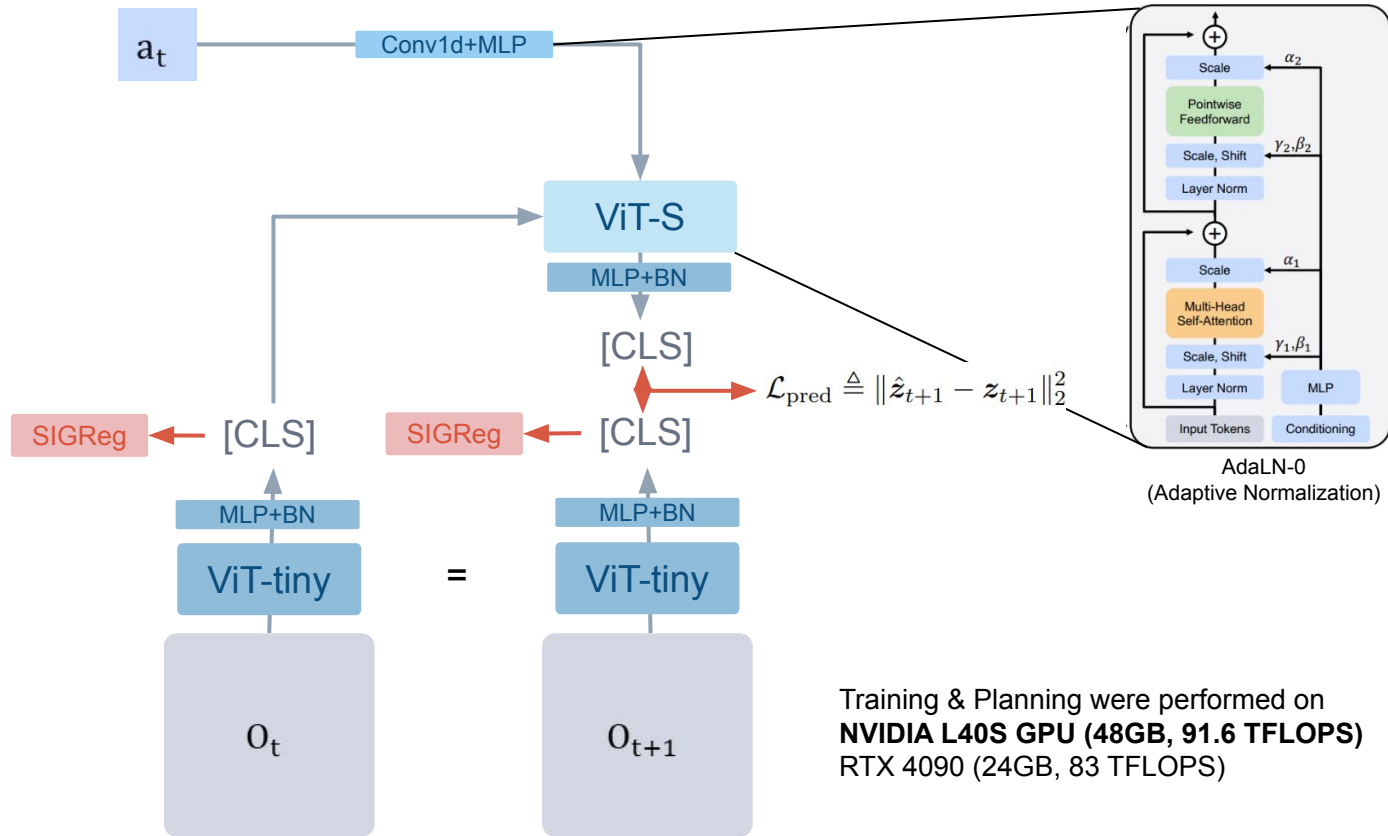
$$\mathcal{L}_{\text{LeWM}} \triangleq \mathcal{L}_{\text{pred}} + \lambda \text{SIGReg}(\mathbf{Z}).$$

Training



현재 관측에서 할
행동을
조건으로 주었을 때,
미래 관측을 예측하는
모델을 학습.

Training

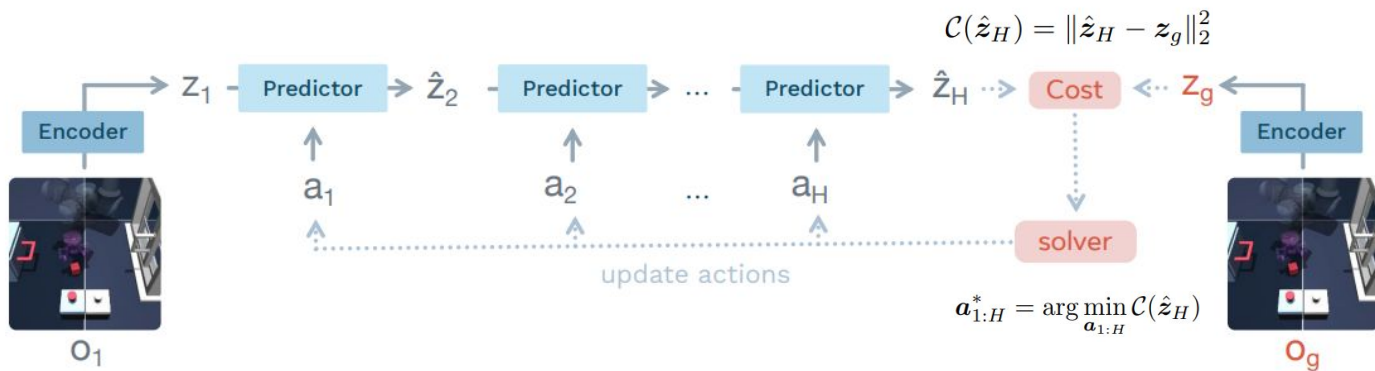


Training & Planning were performed on
NVIDIA L40S GPU (48GB, 91.6 TFLOPS)
 RTX 4090 (24GB, 83 TFLOPS)

$$\mathcal{L}_{LeWM} \triangleq \mathcal{L}_{pred} + \lambda \text{SIGReg}(\mathbf{Z}).$$

Planning

현재 관측에서 타겟 관측으로 가려면
어떤 **action sequence**를 **planning**해야
하는가?



300개의 action sequence 후보, action sequence 뽑기 10번 반복, sequence_len = 5

Contents

1. Introduction
2. Motivation
3. Methods
4. Results
5. Limitations
6. Ideation

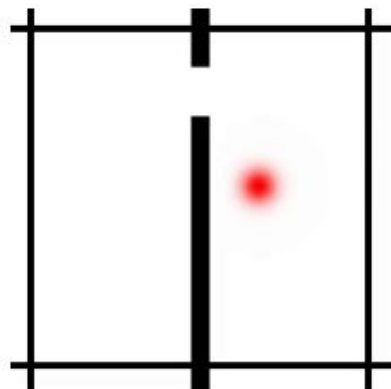
Experiments Setting



Push-T



OGBench-Cube



TwoRoom



Reacher

각 task에 대한 개별 checkpoint가 존재한다.

Planning을 통해 만들어진 길이 5의 action sequence 실행

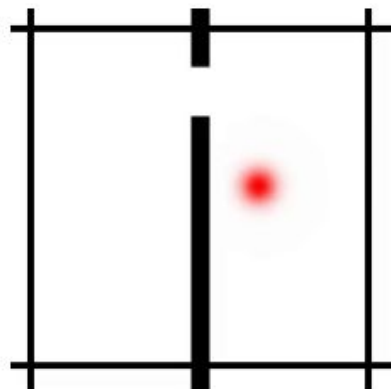
Experiments Setting



Push-T



OGBench-Cube



TwoRoom



Reacher

Push-T: 로봇 end-effector가 T자 모양 물체를 밀어서 목표 자세로 맞추는 manipulation task
20000 episodes, 10 epochs

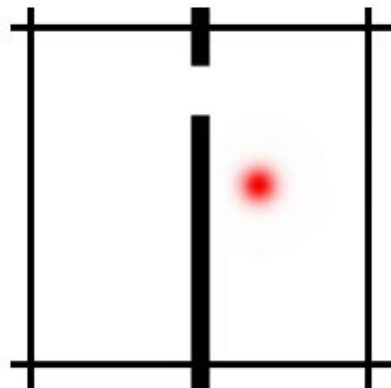
Experiments Setting



Push-T



OGBench-Cube



TwoRoom



Reacher

OGBench-Cube: 로봇이 큐브를 조작해 목표 위치/상태로 옮기는 object manipulation task
10000 episodes, 10 epochs

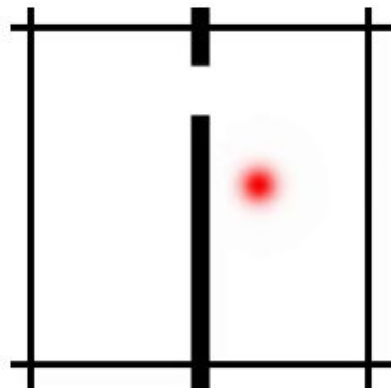
Experiments Setting



Push-T



OGBench-Cube



TwoRoom



Reacher

TwoRoom: 두 방이 있는 2D 미로 환경에서 목표 위치까지 이동하는 navigation task
10000 episodes, 10 epochs

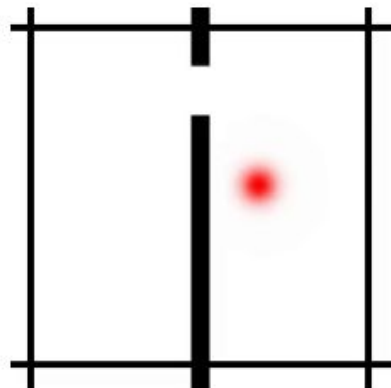
Experiments Setting



Push-T



OGBench-Cube



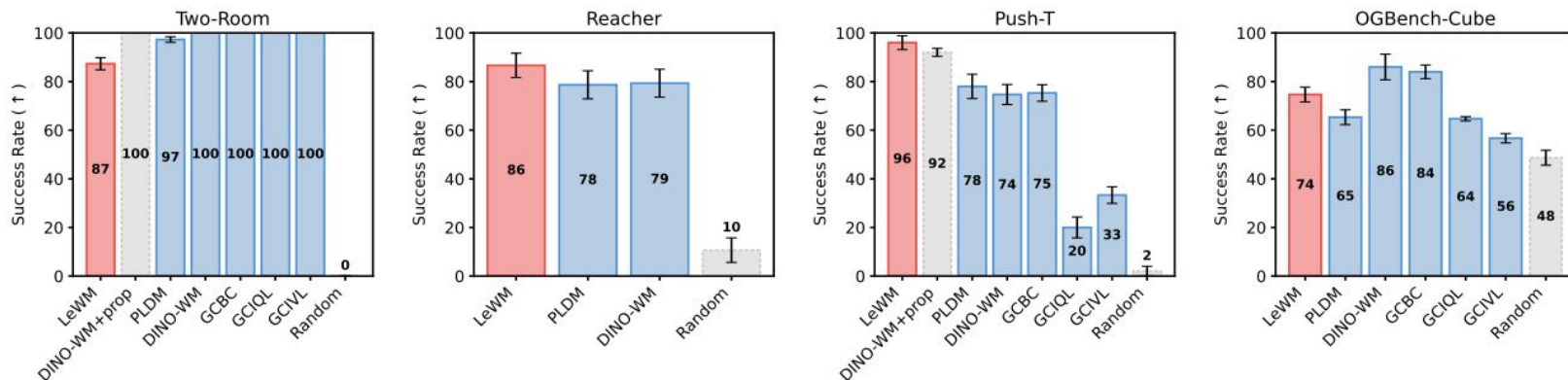
TwoRoom



Reacher

Reacher: 팔이 목표 지점에 end-effector를 가져가는 control task
10000 episodes, 10 epochs

Results (Performance)



*"On OGBench-Cube, DINO-WM slightly outperforms LeWM,
possibly due to the higher visual complexity and the 3D nature of the environment"*

Results (Physical state prediction)

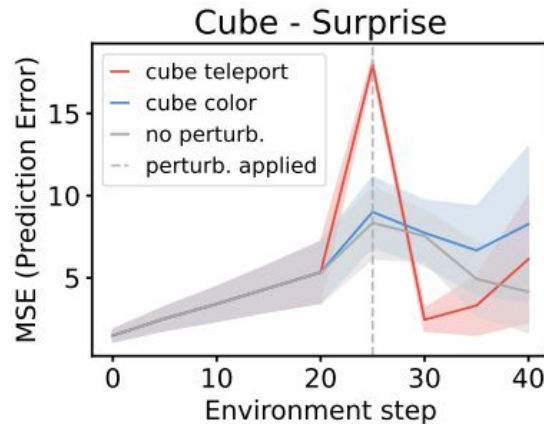
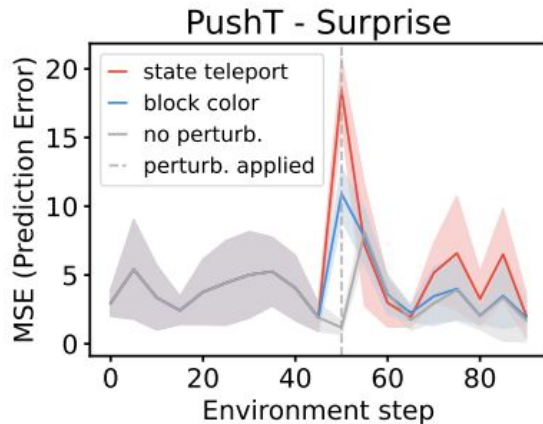
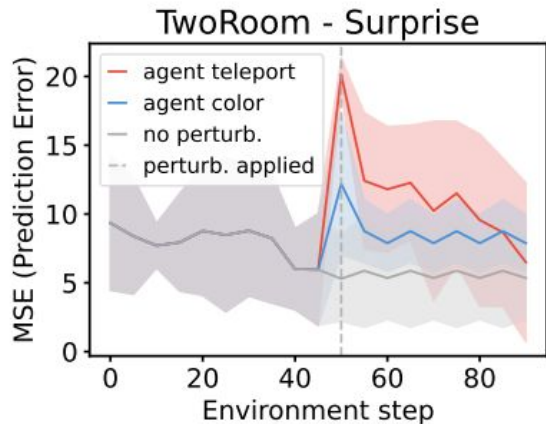
$z_t \rightarrow$ linear layer $\rightarrow$ physical state

PushT		Linear		MLP 피어슨 상관계수	
Property	Model	MSE ↓	r ↑	MSE ↓	r ↑
Agent Location	DINO-WM	1.888 ± 0.500	0.977	0.003 ± 0.022	0.999
	PLDM	0.090 ± 0.311	0.955	0.014 ± 0.119	0.993
	LeWM	0.052 ± 0.149	0.974	0.004 ± 0.056	0.998
Block Location	DINO-WM	0.006 ± 0.007	0.997	0.002 ± 0.003	0.999
	PLDM	0.122 ± 0.341	0.938	0.011 ± 0.066	0.994
	LeWM	0.029 ± 0.073	0.986	0.001 ± 0.006	0.999
Block Angle	DINO-WM	0.050 ± 0.101	0.979	0.009 ± 0.052	0.995
	PLDM	0.446 ± 0.625	0.745	0.056 ± 0.184	0.972
	LeWM	0.187 ± 0.359	0.902	0.021 ± 0.139	0.990

LeWM이 물리 법칙을 이해했다고 보기 어렵다.

Results (Perturbation)

단순 OOD라서 MSE가 튀었다고 볼 수 있는데 왜 이걸 넣은 건지 모르겠다.

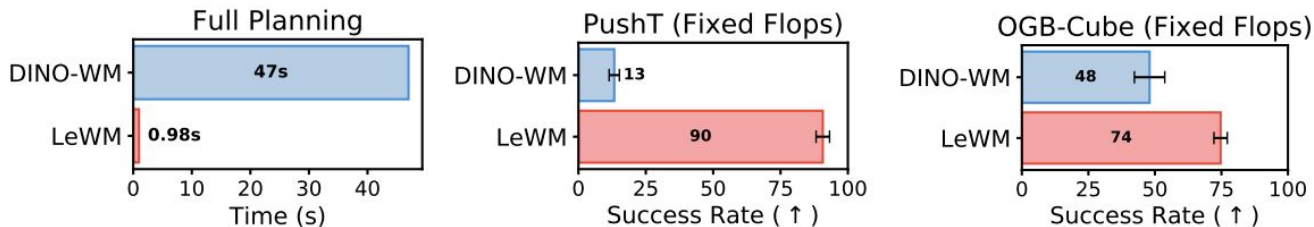


과거 o_t 로 $z_{t+1}^{\wedge}$ 을 예측하게 한 뒤, 실제 다음 o_{t+1} 에 변화가 생기면
teleport와 color change 중 어느 perturbation의 MSE가 튀는지 보는 실험

$$\text{surprise} \approx \|\hat{z}_{t+1} - z_{t+1}\|^2$$

Results (Efficiency)

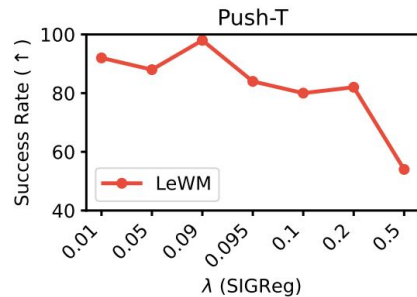
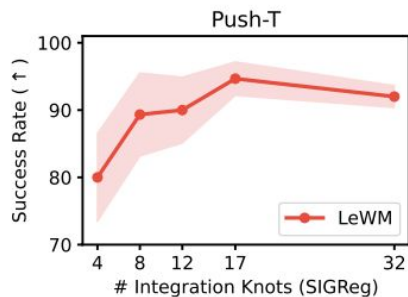
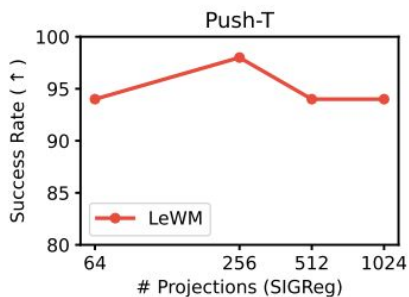
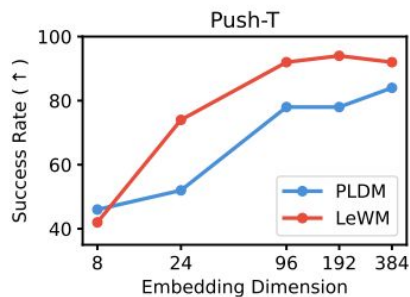
- 6 hyperparameters -> 1 hyperparameter
- Frozen Pre-trained Encoder -> End-to-End Learning
- Task specific -> Task agnostic



Planning time이 DINO-WM 대비 약 48배 단축되었으며,
동일 FLOPs 기준 더 높은 성공률을 가지게 되었다.

Ablations

Hyperparameter가 2개 더 있음;



Embedding Dim & SIGReg hyperparameters ablation

Ablations

Model	Push-T (SR $\uparrow$)
DINO-WM	92.0 ± 1.63
PLDM	78.0 ± 5.0
LeWM (ours)	96.0 ± 2.83

Random seed 3개에 대한 평균 값

pred. size	Push-T (SR $\uparrow$)
tiny	80.67 ± 6.54
small	96.0 ± 2.83
base	86.7 ± 3.06

Predictor ViT size ablation

	Push-T (SR $\uparrow$)
LeWM w/o decoder loss	96.0 ± 2.83
LeWM with decoder loss	86.0 ± 7.54

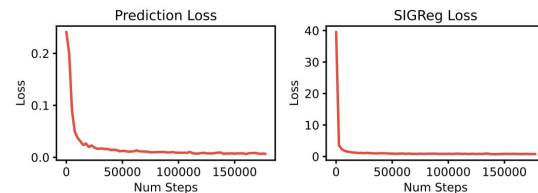
시각화용 decoder loss ablation

	Push-T (SR $\uparrow$)
LeWM ViT	96.0 ± 2.83
LeWM ResNet-18	94.0 ± 3.27

Encoder architecture ablation

p	Push-T (SR $\uparrow$)
0.0	78 ± 6.54
0.1	96.0 ± 2.83
0.2	85.33 ± 5.74
0.5	66.67 ± 4.11

Dropout ablation



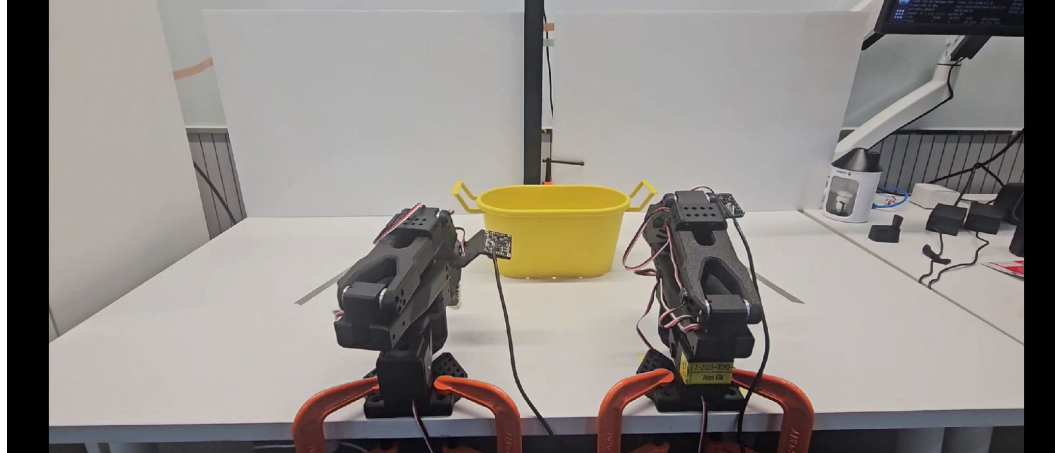
Loss graph

Contents

1. Introduction
2. Motivation
3. Methods
4. Results
5. Limitations
6. Ideation

Limitations of LeWM

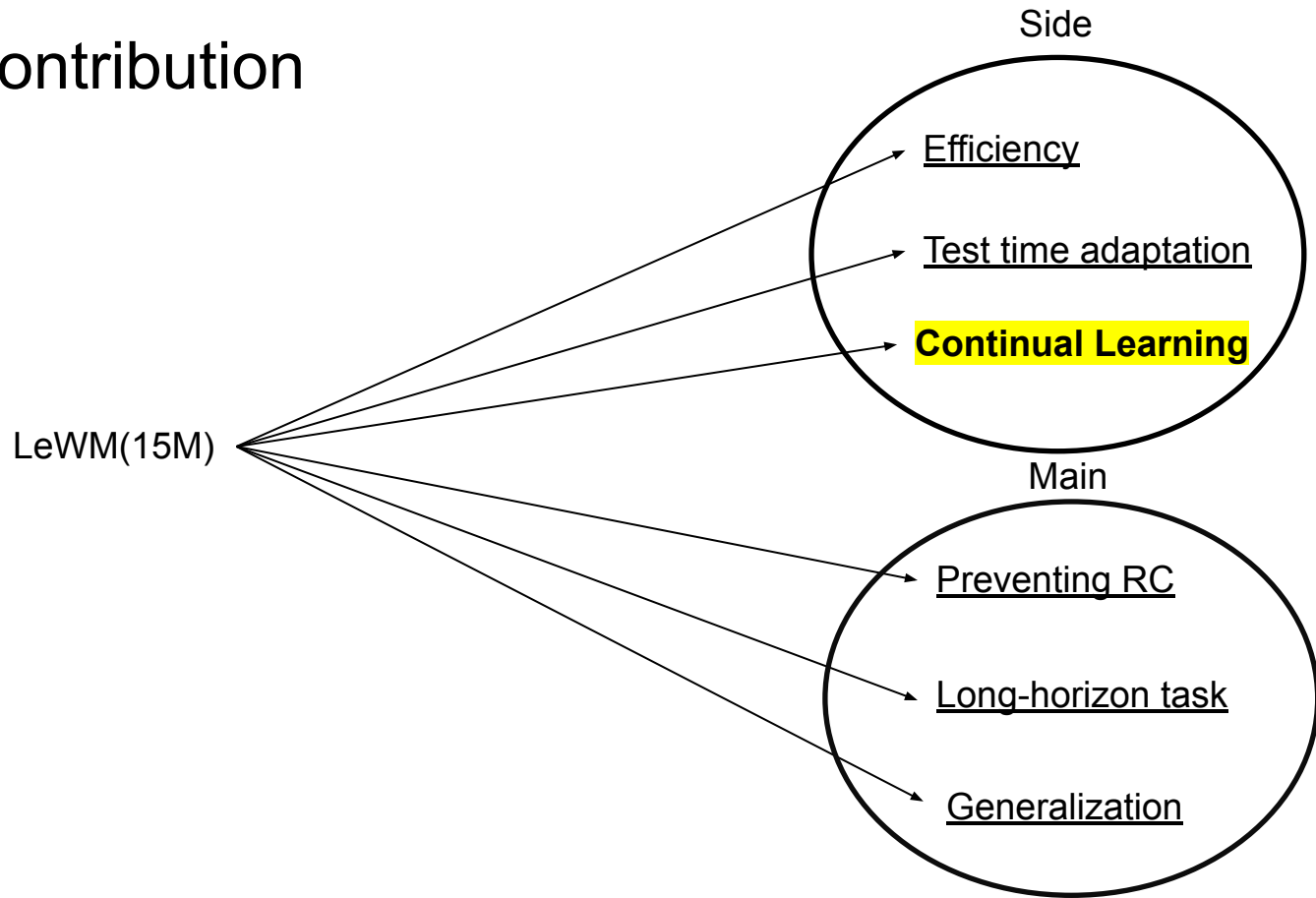
- **Short-Horizon Planning**
- **No Real-World Task**
- Dependence on **Data Diversity**
- Dependence on **Action Labels**



Contents

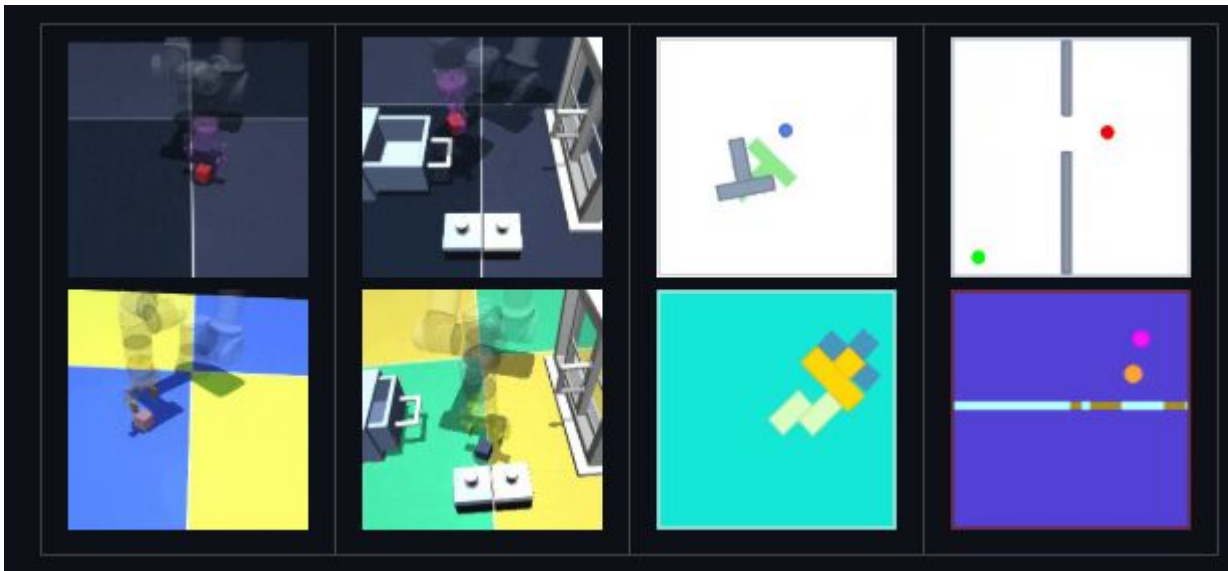
1. Introduction
2. Motivation
3. Methods
4. Results
5. Limitations
6. Ideation

Ways of contribution



실험 환경 구성

- Stable-worldmodel ([paper](#), [repo](#))
- Baselines, Sim Envs 존재



Embodied AI와 Efficient Method에 전반적으로 관심이 있으며

GPU resource를 고려하지 않는 실험은
지양합니다 .

More papers

Efficiency

- Fast LeWorldModel

Long-horizon

- FF-JEPA
- HWM
- SkyJEPA

TTA

- AdaJEPA

Multimodal

- VLJEPA
- LeVLJEPA

Appendix A

How does it make a robot take actions?

MPC

- 주어진 시스템의 동적 모델을 이용해 미래의 거동을 예측하고, 제약 조건을 만족시키면서 비용 함수를 최적화하도록 제어 입력 시퀀스를 계산하는 고급 제어 기법

Appendix B0

Recap DINO DINOv2 DINOv3

Appendix B1

Recap ViT

Appendix C

LeWM Architecture

Appendix D

I-JEPA, V-JEPA

Appendix E

Why is LeWM faster than DINO-WM?

Feature / Metric	DINO-WM	LeWorldModel (LEWM)
Model Type	DINOv2-based Generative World Model	JEPA-based Non-generative World Model
Visual Encoder	DinoV2-S/14 (<code>dinov2_vits14</code>)	ViT-tiny (<code>vit_hf("tiny")</code>)
Latent Representation	Patch Tokens (256 patches) ✂ <i>CLS token also supported</i>	CLS Token Only (<code>output[:, 0]</code>)
Predictor (Transformer)	Depth: 6, Heads: 16, MLP dim: 2048	Depth: 6, Heads: 16, MLP dim: 2048
Total Parameters	~40M	~15M

Appendix F

How about turn off the SIGReg?

```

class SIGReg(torch.nn.Module):
    """Sketch Isotropic Gaussian Regularizer (single-GPU!)"""

    def __init__(self, knots=17, num_proj=1024):
        super().__init__()
        self.num_proj = num_proj
        t = torch.linspace(0, 3, knots, dtype=torch.float32)
        dt = 3 / (knots - 1)
        weights = torch.full((knots,), 2 * dt, dtype=torch.float32)
        weights[[0, -1]] = dt
        window = torch.exp(-t.square() / 2.0)
        self.register_buffer("t", t)
        self.register_buffer("phi", window)
        self.register_buffer("weights", weights * window)

```

```

def forward(self, proj):
    """
    proj: (T, B, D)
    """

    # 1. 무작위 프로젝션 벡터 샘플링 (Random Projection)
    A = torch.randn(proj.size(-1), self.num_proj, device=proj.device)
    A = A.div_(A.norm(p=2, dim=0))

    # 2. Epps-Pulley 정규성 통계량 계산
    x_t = (proj @ A).unsqueeze(-1) * self.t
    err = (x_t.cos().mean(-3) - self.phi).square() + x_t.sin().mean(-3).square()
    statistic = (err @ self.weights) * proj.size(-2)

    # 3. 시간 및 무작위 투영 공간 전체에 대한 평균 반환
    return statistic.mean() # average over projections and time

```

(Press **h** for help or **q** to quit)

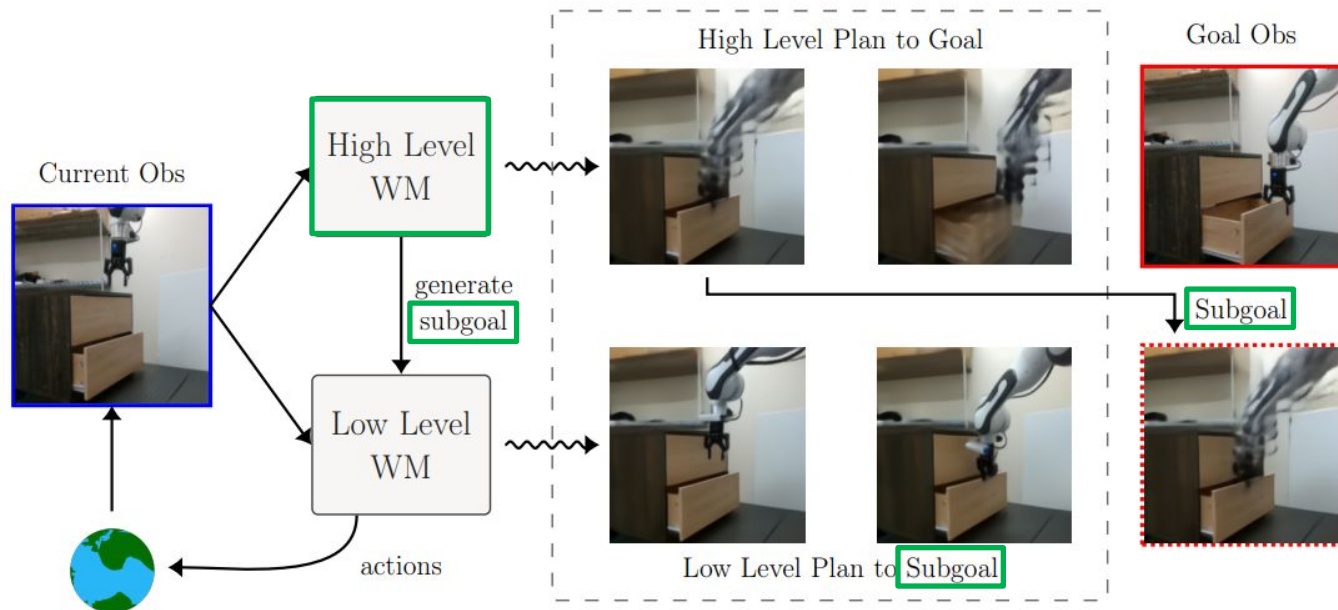
NVITOP 1.7.0		Driver Version: 581.80		CUDA Driver Version: 13.0	
GPU Fan Temp Perf Pwr:Usg/Cap		Memory-Usage		GPU-Util	Compute M.
0 N/A 78C P0 60W / N/A		6079MiB / 8188MiB		100%	Default
		MEM: 6079MiB			
[CPU: 14.4%		[MEM: 70.0%		[UPTIME: 1.6 days]	
				[Load Average: 0.00 0.00 0.00]	
				[SWP: 13.0%	

Embedding Standard Deviation over Training Steps (15M Model)

The graph plots Embedding Standard Deviation (Y-axis, 0.0 to 1.0) against Training Steps (X-axis, 0 to 150). It compares two configurations: SIGReg Off ($\lambda = 0.0$) and SIGReg On ($\lambda = 0.09$), with a Collapse Threshold at 0.15.

- SIGReg Off ($\lambda = 0.0$):** Represented by a red line with circular markers. It starts at ~0.25, peaks at ~0.7 at step 30, and then drops sharply to ~0.18 at step 45, remaining stable thereafter.
- SIGReg On ($\lambda = 0.09$):** Represented by a green line with square markers. It starts at ~0.25, rises steadily to ~0.95 by step 40, and then fluctuates slightly around 0.95-1.0.
- Collapse Threshold (0.15):** Represented by a horizontal yellow dotted line at y = 0.15.

Training Steps	SIGReg Off ($\lambda = 0.0$)	SIGReg On ($\lambda = 0.09$)	Collapse Threshold (0.15)
0	0.25	0.25	0.15
2	0.28	0.35	0.15
4	0.28	0.40	0.15
6	0.28	0.45	0.15
15	0.42	0.72	0.15
30	0.70	0.92	0.15
45	0.18	0.94	0.15
60	0.12	0.97	0.15
75	0.10	0.93	0.15
90	0.09	0.96	0.15
105	0.08	0.95	0.15
120	0.08	0.93	0.15
135	0.08	0.99	0.15
150	0.08	1.00	0.15



Hierarchical Planning with Latent World Models

Meta, 2026.04

What are the **limitations** of HWM?

Can we **contribute** to this subject?

Contents

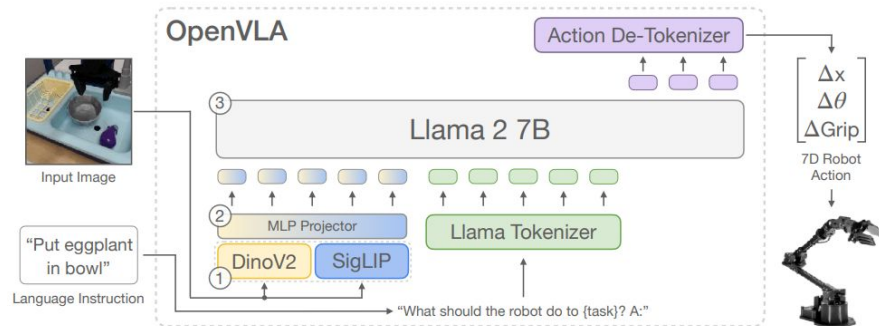
- Introduction
- Motivation
- Methods
- Results
- Thought

Introduction

Why World Model?

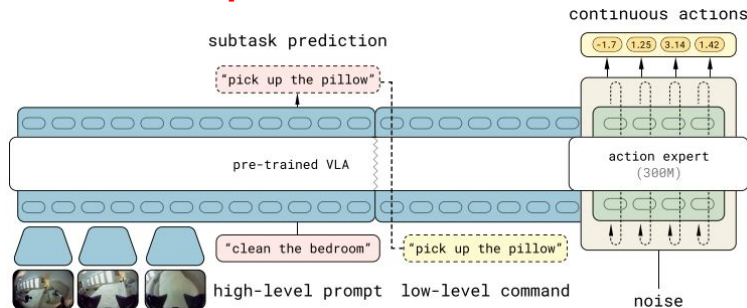
"Learn the world, not the task — then new tasks become planning problems."

VLA

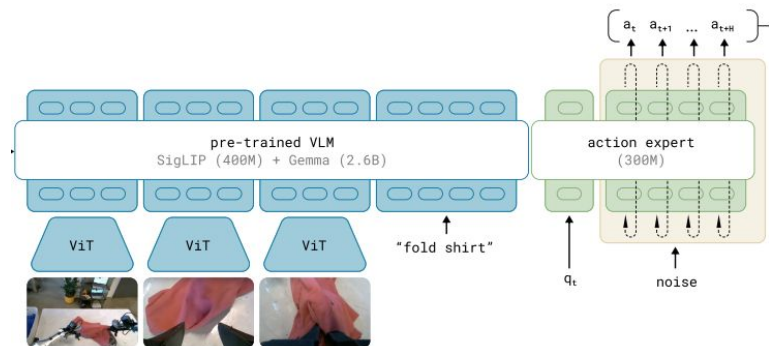


OpenVLA
2024.07

Do not predict the future frame



Pi_0.5
2025.04

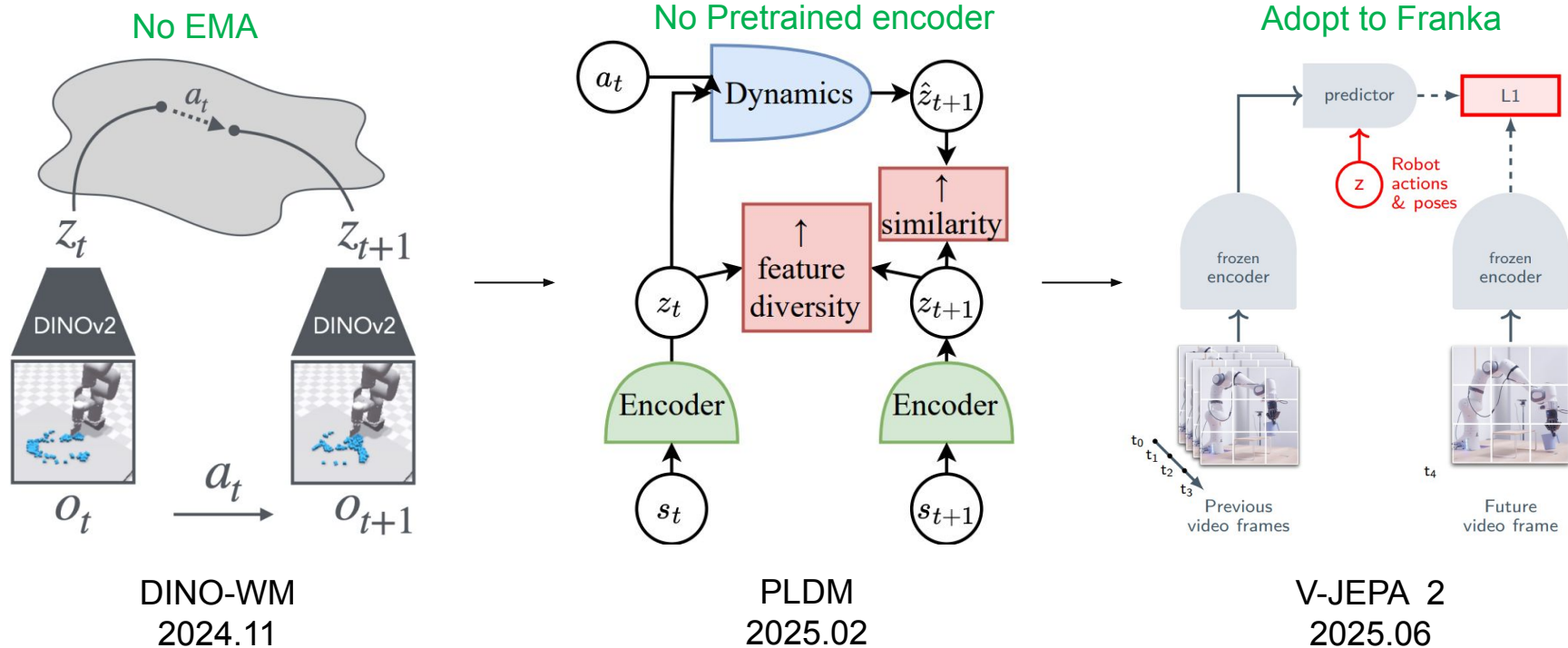


Pi_0
2024.10



Latent World Model Planning

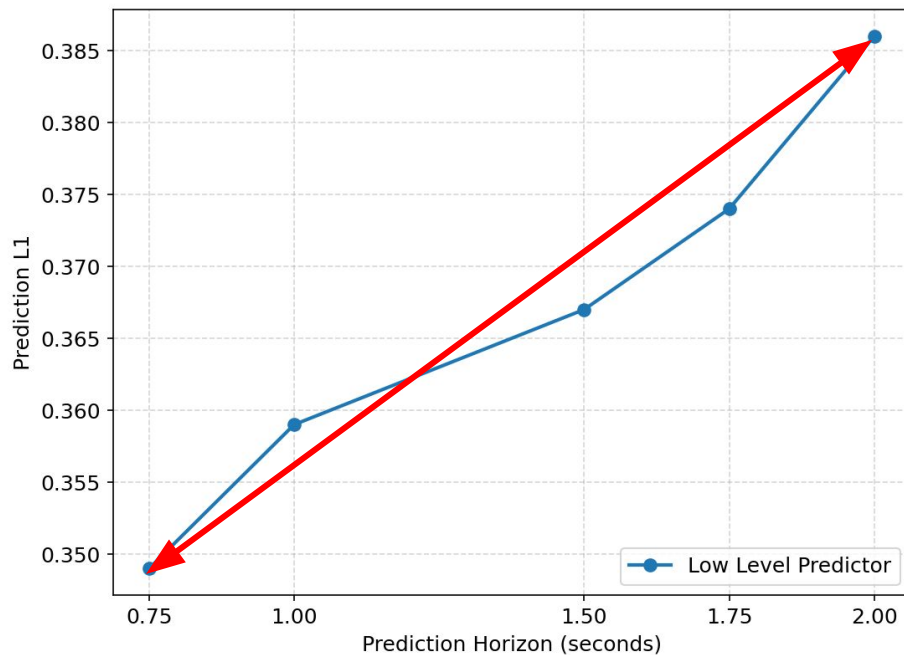
Only for short-horizon greedy tasks



Motivation

“However, existing world model planners struggle on
long-horizon, multi-stage tasks”

Long-horizon Weakness



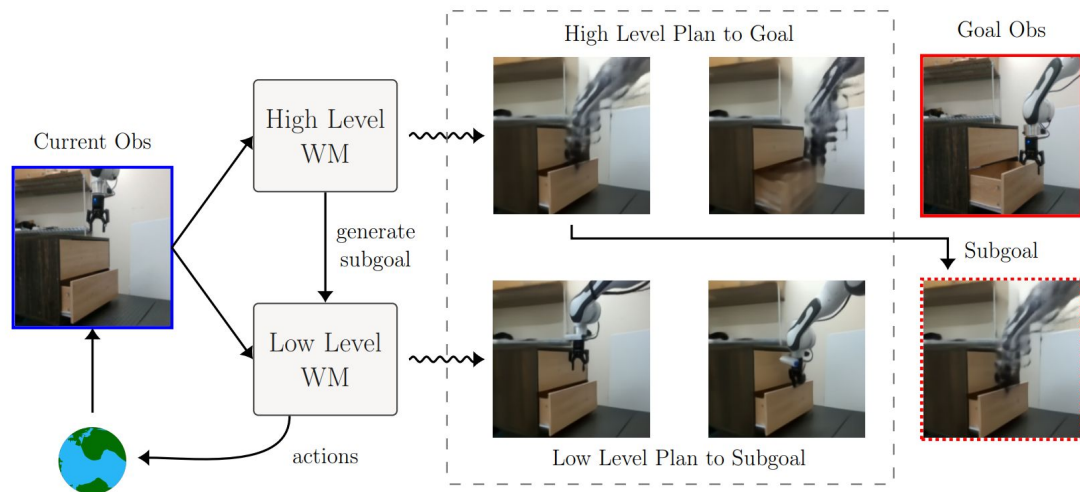
**Compounding error
of a low-level predictor**

Multi-stage(Non-greedy) Weakness

Method	P&P Cup	P&P Box	Drawer
VJEPA2-AC	0%	0%	30%

Methods

Training & Planning



Training

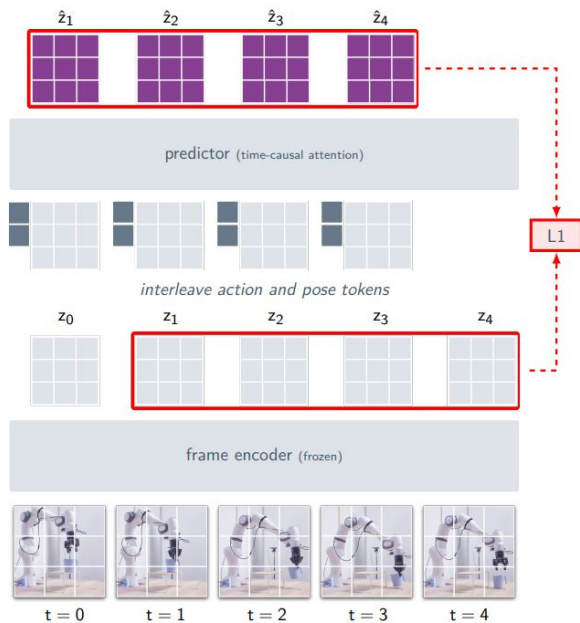
Method (Training)



Training Separately

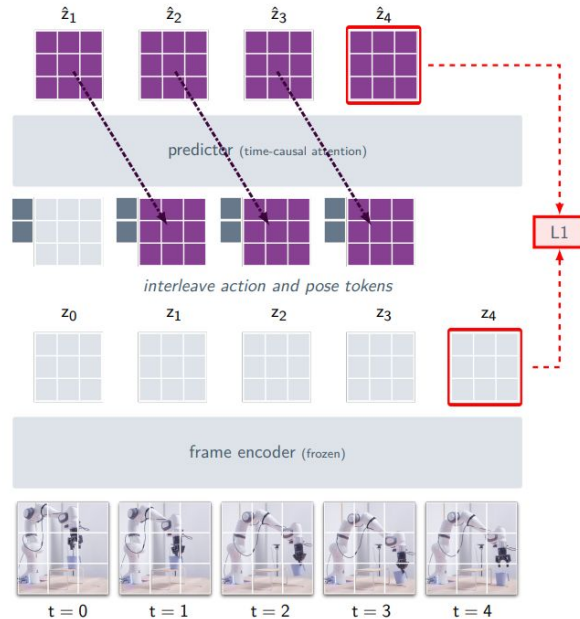
Method (Training F^1)

VJEPA 2-AC Teacher Forcing Loss



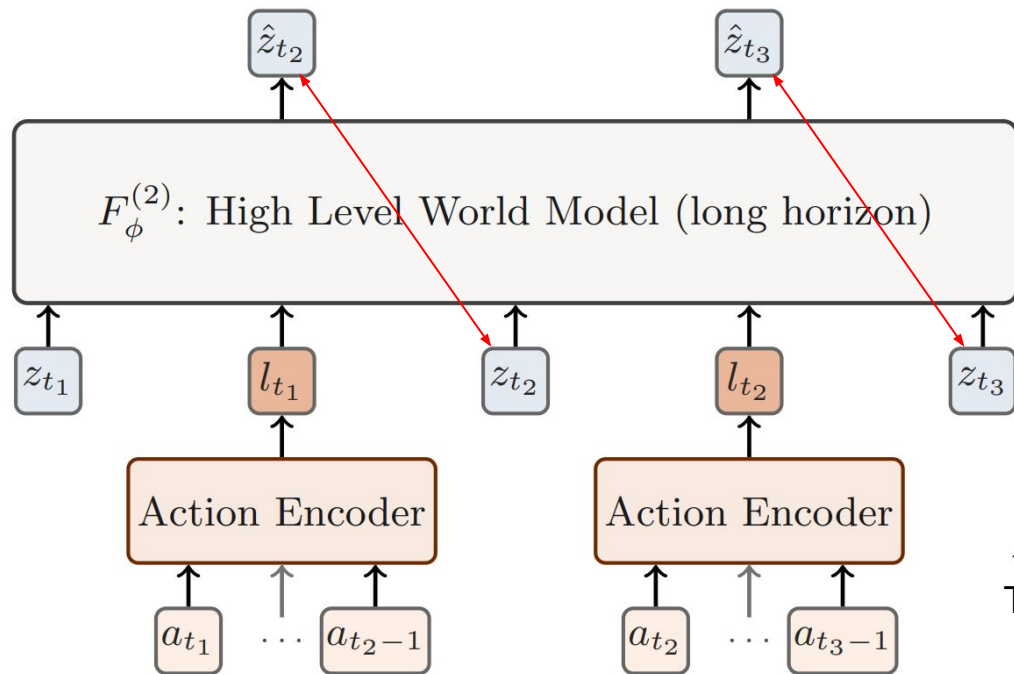
$$\mathcal{L}_{\text{tf}}(\theta) := \frac{1}{T} \sum_{k=1}^T \|\hat{z}_{k+1} - z_{k+1}\|_1.$$

VJEPA 2-AC Rollout Loss



$$\mathcal{L}_{\text{roll}}(\theta) := \sum_{j=2}^T \|F_{\theta}^{(1)}(a_{1:j}, z_1) - z_{j+1}\|_1$$

Method (Training F^2)



$$\mathcal{L}_{\text{tf}}(\phi, \psi) := \frac{1}{N} \sum_{k=1}^N \|\hat{z}_{t_{k+1}} - z_{t_{k+1}}\|_1$$

Teacher forcing loss

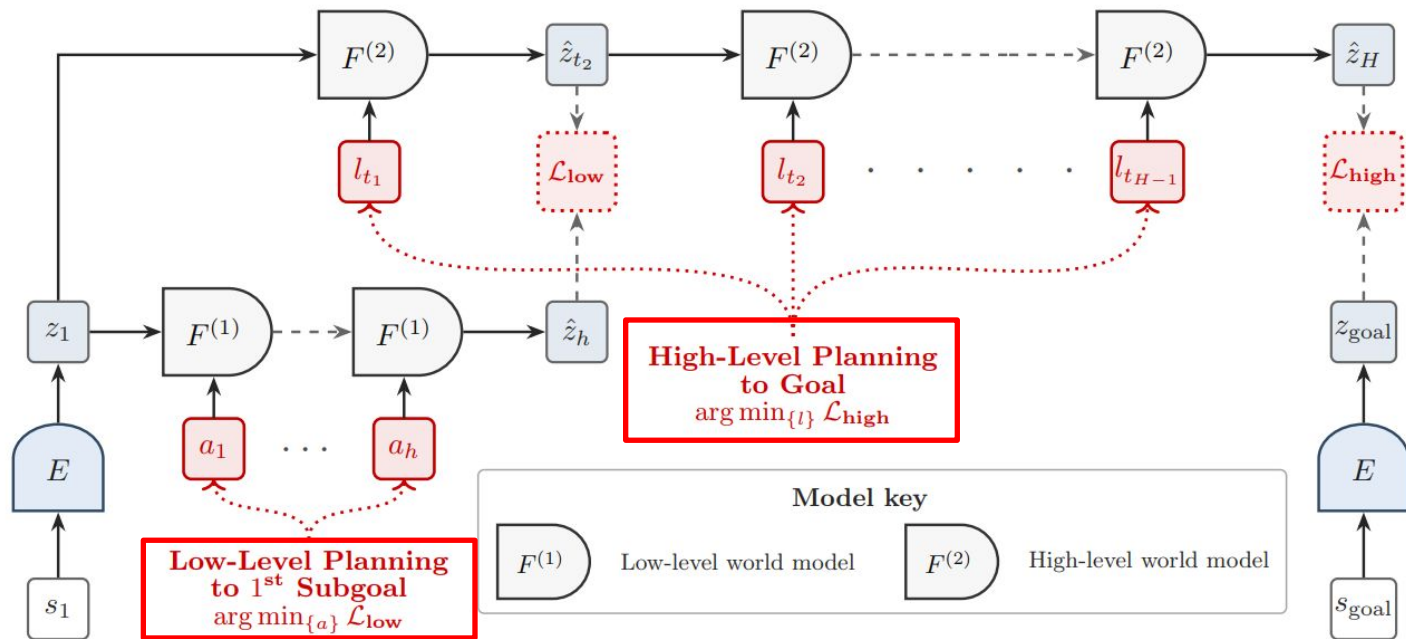
Method (Training Details)

World Model	Epochs	Batch Size	γ_{tf}	γ_{roll}	context T
VJEPA2-AC (Low-Level)	200	256	1.0	1.0	16
VJEPA2-AC (High-Level)	120	768	1.0	0.0	3

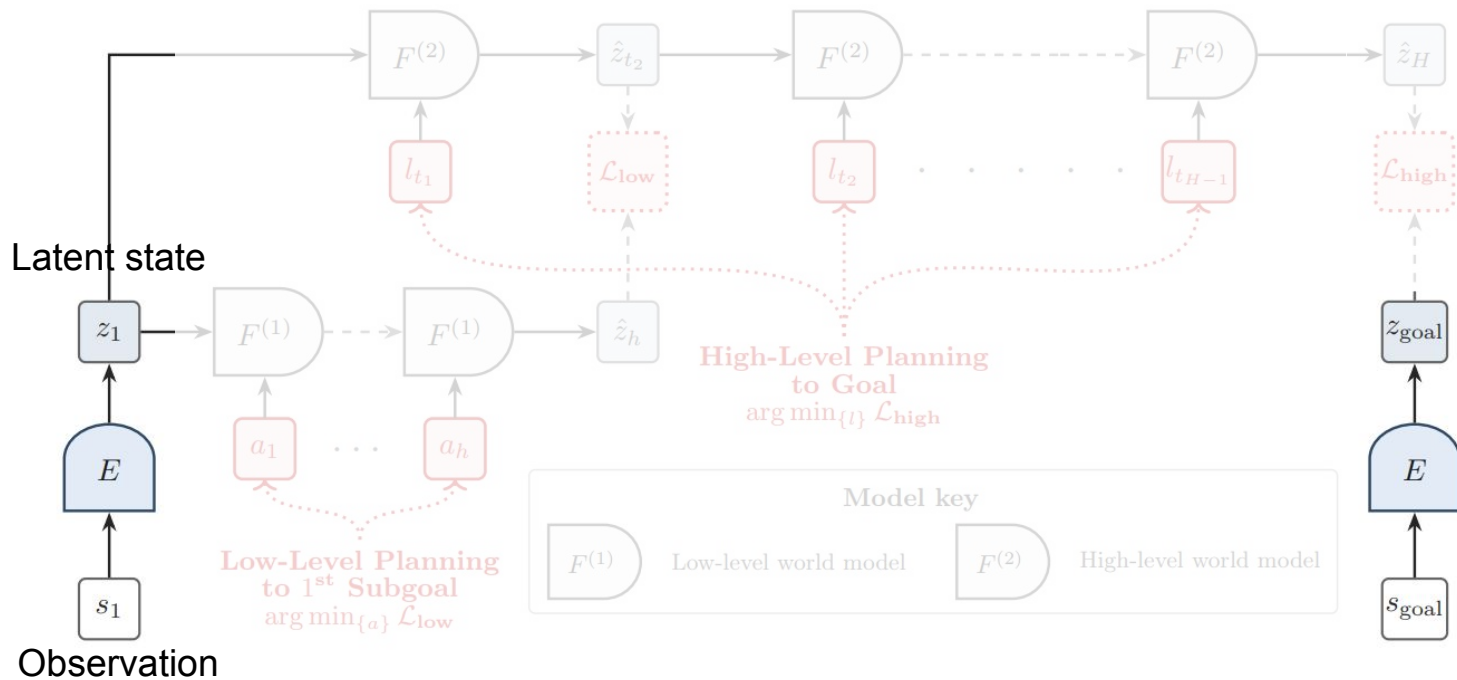
Used **DROID** (96 hours) & **RoboSet** (30 hours)

Planning

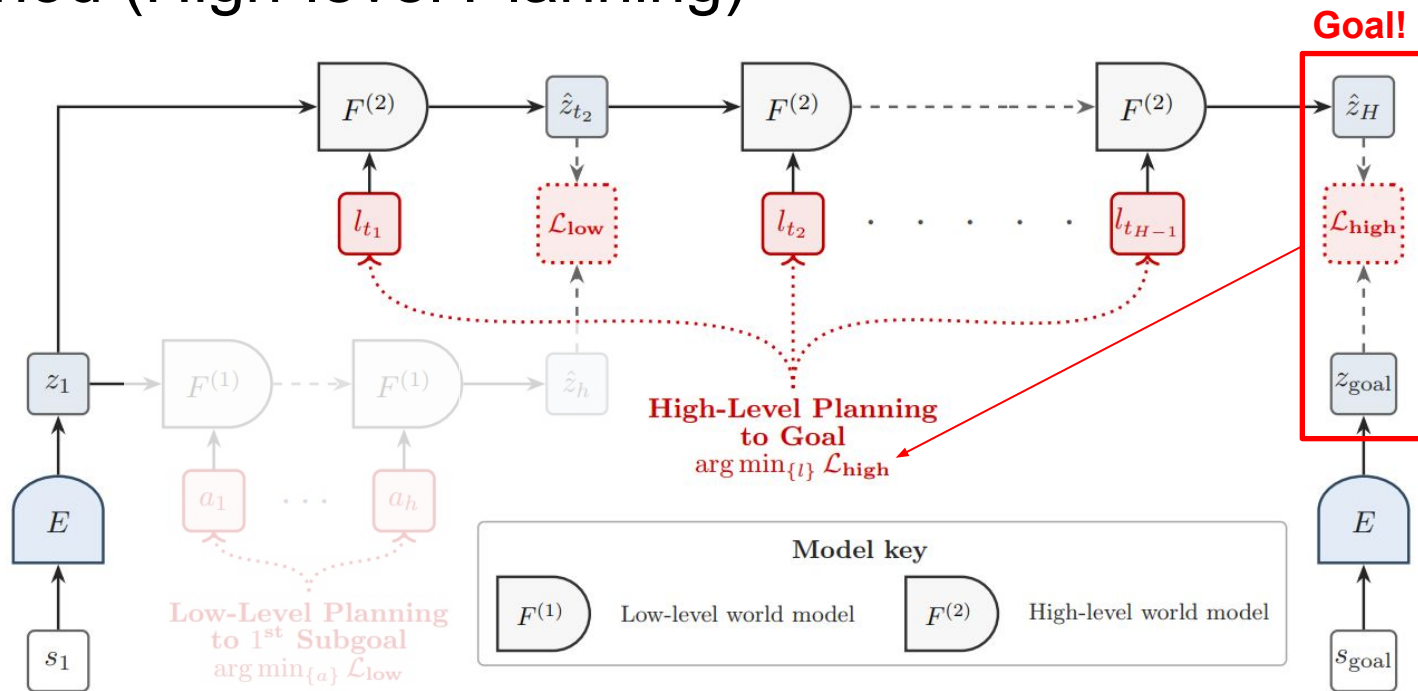
Method (Planning)



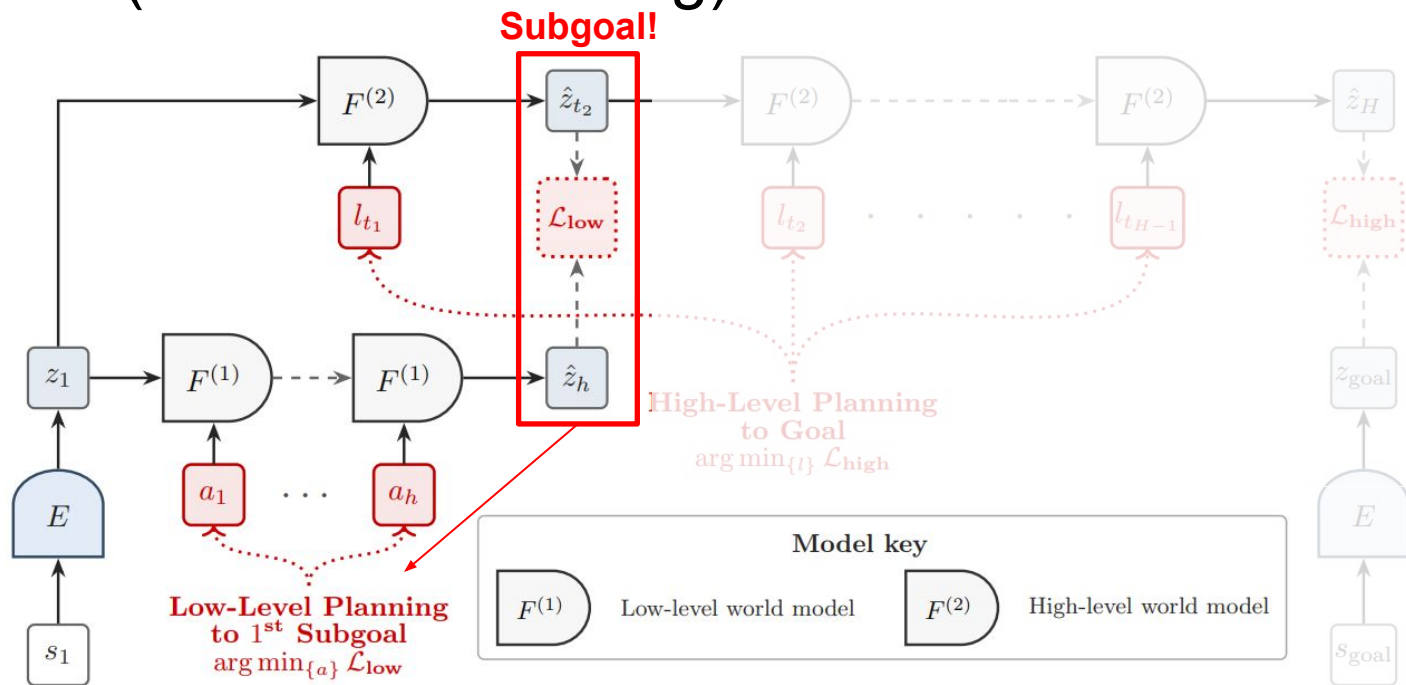
Method (Encoding)



Method (High-level Planning)



Method (Low-level Planning)



Method (Planning: MPC)



Input: current state s_t , goal s_g , world model F , planning horizon h , execution steps k

Repeat:

1. Optimize candidate action sequence: $\hat{a}_{t:t+h-1}$

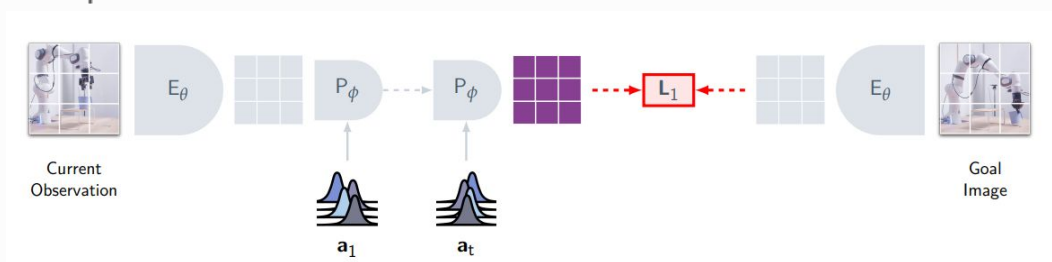
$$\hat{a}_{1:h}^* = \arg \min_{\hat{a}_{1:h}} \|z_g - F(\hat{a}_{1:h}; z_1)\|_1$$

(roll out F autoregressively for h steps, **evaluate cost against the goal**)

2. **Execute only the first k actions** ($k \leq h$) in the real environment

3. Observe **the new state** s_{t+k}

4. Return to step 1 and replan



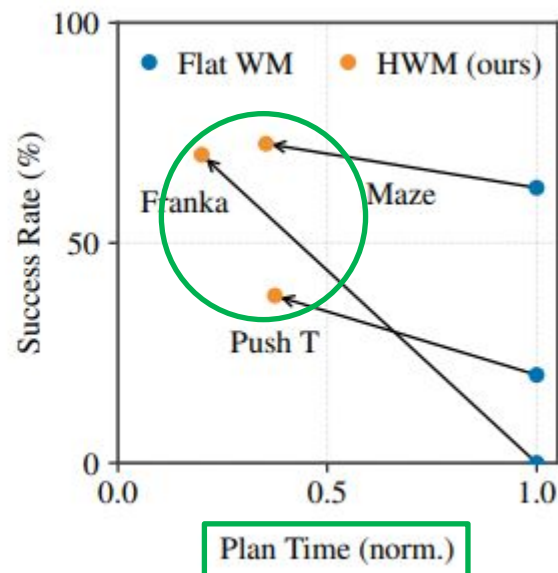
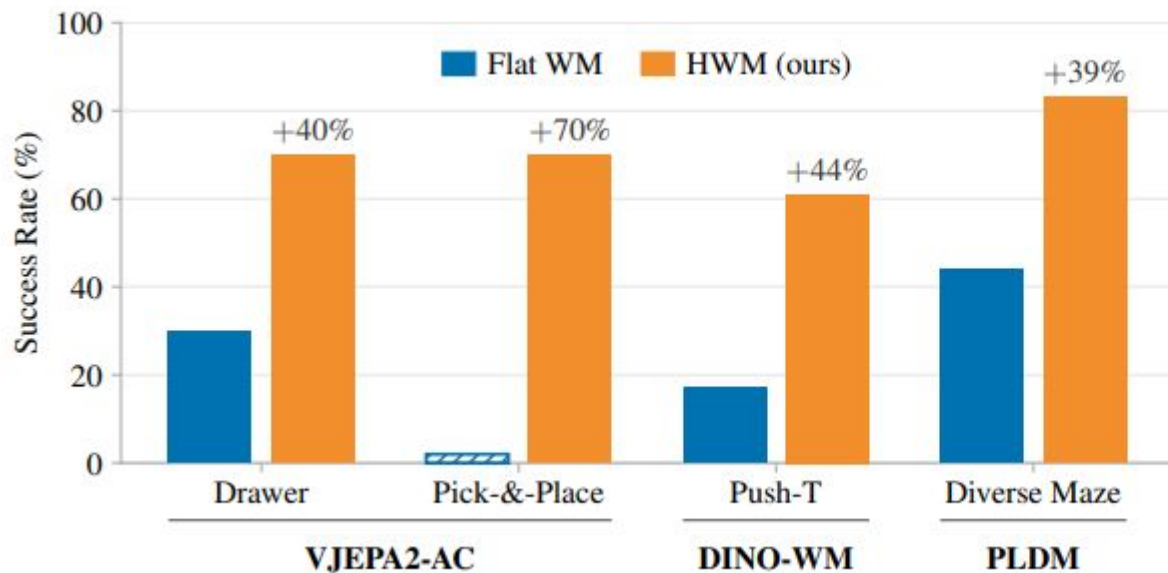
Method (Planning Details)

	High-level planner					Low-level planner					
	# elites	# iters	# samples	Var EMA	pred H	# elites	# iters	# samples	Var EMA	pred h	k
Franka	22	15	3000	0.65	2	12	5	800	0.25	2	1
Push-T($d = 25$)	10	20	900	0	2	10	30	300	0	5	5
Push-T($d = 50$)	10	40	1500	0.9	4	10	20	900	0.8	5	5
Push-T($d = 75$)	10	20	1200	0.9	5	10	20	1200	0.8	5	5

Planner Hyperparameter Grid Search

Results

High Performance!



Handled non-greedy manipulation!

Non-greedy task!

Method	P&P Cup	P&P Box	Drawer
Octo	0%	0%	43%
π_0 -FAST-DROID	52%	18%	—
$\pi_{0.5}$ -DROID	68%	36%	—
VJEPA2-AC	0%	0%	30%
VJEPA2-AC (hierarchy)	 70%	 60%	70%

Zero-shot setting

Improved Performance on Long-Horizon Tasks!

Method	$d = 25$	$d = 50$	$d = 75$
GCIQL	40%	25%	7.5%
HIQL	55%	30%	20%
HILP	25%	13%	0%
DINO-WM	84%	55%	17%
DINO-WM (hier.)	89%	78%	61%

Long horizon task!

Used Push-T

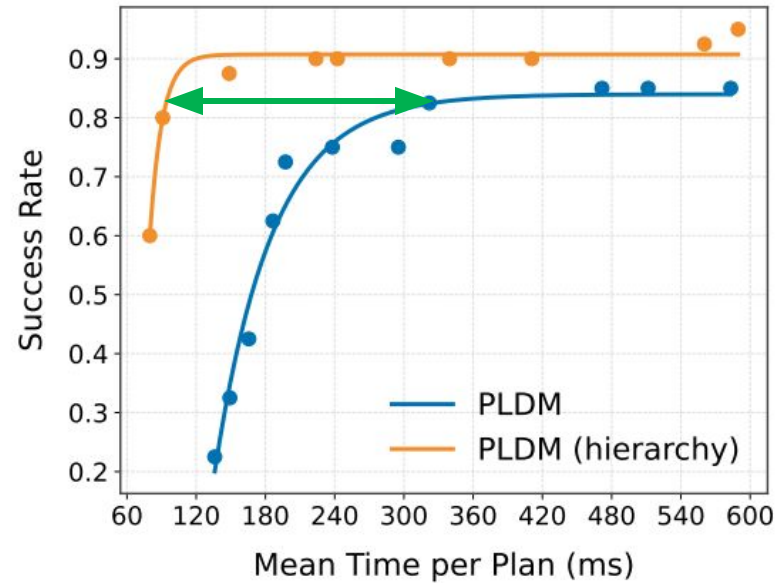
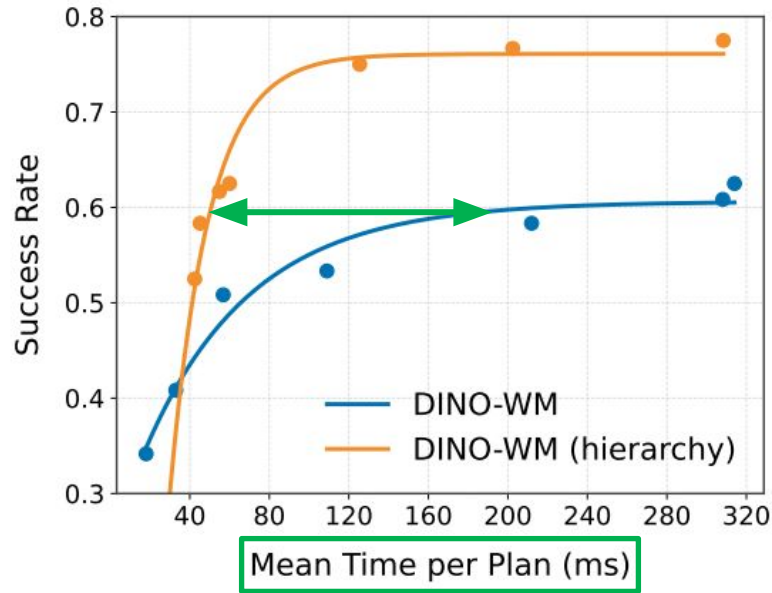
Improved Performance on Long-Horizon Tasks!

Method	$D \in [5, 8]$	$D \in [9, 12]$	$D \in [13, 16]$
GCIQL	85%	40%	33%
HIQL	88%	73%	48%
HILP	48%	20%	10%
PLDM	100%	84%	44%
PLDM (hier.)	100%	95%	83%

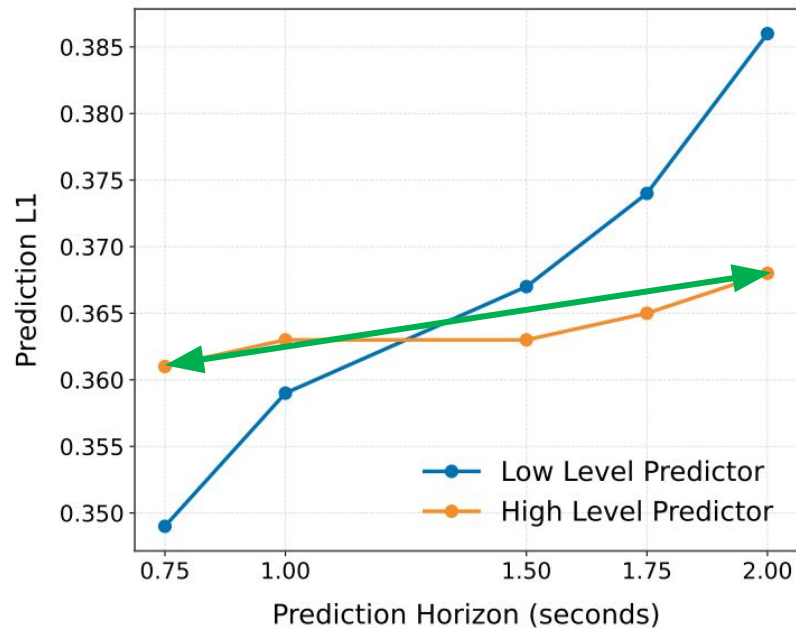
Long horizon task!

Used Diverse Maze

Better Performance with Less Planning Compute!



Compounding Error Mitigated!



Why do they use an action encoder?

High Level Action	$D \in [9, 12]$	$D \in [13, 16]$
Concat Primitive Actions	52	37
Macro-Action	95	83

High Level Action	Cos $\uparrow$	$\ell_1 \downarrow$
Delta Pose	0.80 ± 0.02	0.088 ± 0.005
Macro-Action	0.88 ± 0.03	0.080 ± 0.002

Model Size vs. Hierarchy

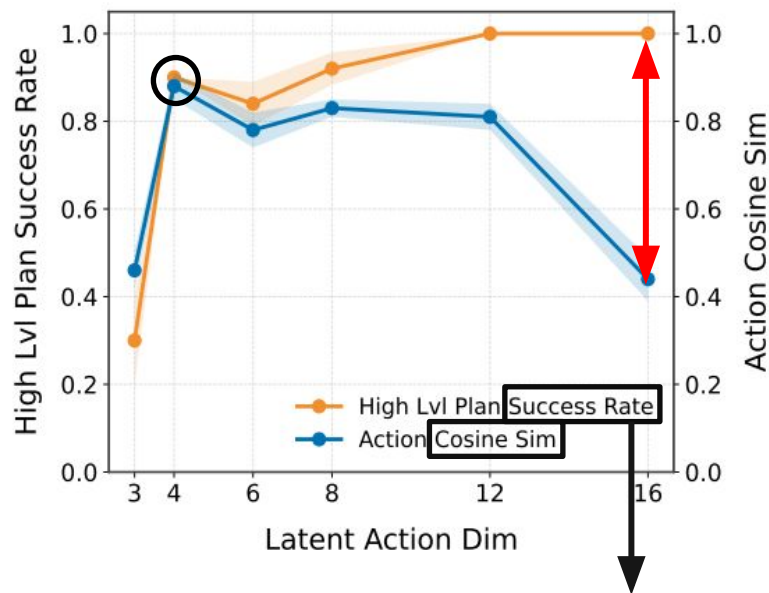
Method	Params	$H=10$	$H=15$
DINO-WM (flat)	44M	55%	17%
DINO-WM (flat)	98M	35%	15%
DINO-WM (hierarchy)	94M	78%	61%

(a) Push-T

Method	Params	$D \in [9, 12]$	$D \in [13, 16]$
PLDM (flat)	54k	85%	63%
PLDM (flat)	178k	82%	59%
PLDM (hierarchy)	182k	95%	83%

(b) Diverse Maze

Valid Plans $\neq$ Executable Plans: The Macro-Action Dimension Trade-off



**Rate of decoded latent predictions
that match the goal state**

Limitations

- No reachability guarantee for subgoals.
- Ungrounded macro-action space.
- Heuristic waypoint selection.

Appendix

A. Experimental Setup

EXPERIMENTAL SETUP

세 가지 latent world model 백본 위에 HWM을 인스턴스화하고, 각 원 논문보다 어려운 조건에서 평가

Franka (실물 로봇)

VJEPA2-AC

원 세팅 대비 난이도 ↑

명시적 subgoal 제거

단일 goal image만 제공

Pick-&-place · Drawer

P&P 0% → 70%

Push-T (시물)

DINO-WM

원 세팅 대비 난이도 ↑

Horizon 25 → 75 스텝 확장

Goal-conditioned push

d=75: 17% → 61%

Diverse Maze (시물)

PLDM

원 세팅 대비 난이도 ↑

학습에 없던 더 큰 레이아웃

(train-test mismatch)

PointMaze 내비게이션

$D \in [13,16]$: 44% → 83%

공통: 평가 지표는 binary success rate · 디코더 재구성 이미지는 시각화 전용 (planning/training 미관여)

Franka 실물 로봇 — VJEPA2-AC 백본

태스크 · 평가 프로토콜

플랫폼

7-DoF Franka Panda + Robotiq 2-finger 그리퍼

구성

Pick-&-place 10구성 × 2물체(컵/박스) · Drawer open/close 7구성

Trial

구성당 N=5 → 물체당 50, drawer 35 trial

Goal 지정

단일 goal image (RGB + EE pose), 중간 subgoal 없음

데이터 · 모델

데이터

DROID 96h + RoboSet 30h (~130h, 무라벨) · 액션 = EE delta pose

Low-level WM

Frozen ViT-g/16 인코더 + ~300M ViT, T=15

High-level WM

0.33-4초 구간, N=3 waypoint(중간점 랜덤) · 4-D macro-action

Planner

양 레벨 CEM, 매 스텝 replan (k=1)

0% → 70%

P&P cup, flat 대비

Baseline: Flat VJEPA2-AC (동일 아키텍처·데이터 → hierarchy 효과 분리)
· VLA: Octo, π_0 -FAST-DROID, $\pi_{0.5}$ -DROID (zero-shot)

Push-T — DINO-WM 백본

태스크 · 데이터

태스크

T자 물체를 goal configuration까지 push · start-goal은 validation trajectory에서 샘플

Horizon

d = 25 / 50 / 75 스텝 (원 논문은 d=25까지만 평가)

데이터

DINO-WM 공개셋 18,500 trajectories (RGB + 2D action + proprio)

모델 · Planning

Low-level WM

Frozen DINOv2 + 25M causal ViT · stride 5 (유효 horizon 25스텝) · 액션 = primitive 5개 concat (10-D)

High-level WM

75M ViT (6→10 layers, dim 384→768) · 25-70스텝 구간에서 N=5 waypoint · 4-D macro-action

Planner

2단계 CEM, k=5로 replan

17% → 61%

d=75, flat 대비

Baseline: Flat DINO-WM · HIQL, GCIQL, HILP (난이도별 하이퍼파라미터 개별 튜닝) · capacity-matched flat (98M)도 hierarchy 미달

Diverse Maze — PLDM 백본

태스크 · 데이터

환경

MuJoCo PointMaze 변형 · 98×98×3 top-down RGB
입력 · 8×8 grid 랜덤 레이아웃

OOD 분리

학습 25맵 / 평가 20 held-out맵 · 난이도 $D \in [5, 8] / [9, 12] / [13, 16]$

데이터

맵당 2,000 에피소드 (랜덤 액션 100스텝, action repeat 4) → 5M transitions

모델 · Planning

Low-level WM

Conv 인코더(33k) + predictor(20k) scratch 공동학습 ·
15-step rollout · VICReg으로 collapse 방지

High-level WM

Frozen low-level 인코더 재사용 · 고정 stride 10,
waypoint 6개 · 8-D macro-action (MLP)

Planner

2단계 MPPI, k=4로 replan

44% → 83%

$D \in [13, 16]$, flat 대비

Baseline: Flat PLDM · HIQL, GCIQL, HILP · RL 계열은 OOD
레이아웃에서 급락 → model-based planning의 분포 이동 강건성 부각

DETAILS

학습 · Planning 하이퍼파라미터

학습

항목	Franka	Push-T	Maze
Low epochs / batch	200 / 256	100 / 256	3 / 128
High epochs / batch	120 / 768	500 / 128	5 / 128
Low loss (ytf, yroll)	1.0, 1.0	1.0, 0	0, 1.0
High loss (ytf, yroll)	1.0, 0	1.0, 0	0, 1.0
Waypoint	N=3, 0.33-4s, 랜덤	N=5, 25-70스텝	stride 10 고정, 6개

Planning (hierarchical)

항목	Franka	Push-T (d=75)	Maze ($D \in [13, 16]$)
알고리즘 / replan k	CEM / 1	CEM / 5	MPPI / 4
High: samples · iters · H	3000 · 15 · 2	1200 · 20 · 5	4000 · - · 47
Low: samples · iters · h	800 · 5 · 2	1200 · 20 · 5	1000 · - · 15

분석 실험 세팅 (Section 4)

4.1

Manual subgoal 대조

P&P 각 에피소드에 grasp 지점 subgoal 이미지를 수동 부여 → flat도 80%/80% 달성. 실패 원인이 표현이 아닌 planning임을 분리

Flat 0% → 80% (oracle)

4.2

장기 예측 정확도

DROID held-out에서 최대 2초 예측 ℓ_1 비교. Low-level은 최대 16 autoregressive step vs high-level 단일 step

$\geq 1.5s$ 에서 high-level 우세

4.3

Macro-action 표현 비교

학습된 macro-action vs primitive concat vs delta pose(handcrafted). Maze 성공률 및 Franka expert 액션 정렬도(cos, ℓ_1)로 평가

Maze 52% → 95% (concat 대비)

4.4

Macro-action 차원 ablation

$d \in \{3, 4, 6, 8, 12, 16\}$ 스윙. Plan validity(정성) + 첫 subgoal 도달 가능성(expert cos sim)의 trade-off 분석

Franka 최적 $d=4$